Reflection

What I'd build differently with 3 months instead of 8 hours

- **The LLM Mapper end to end, with mapping memory.** Approved mappings would be stored per client and per ERP, so a decision made in Q4 is reused in Q1. The LLM would only see genuinely new accounts, and every reuse would be tracked as a versioned mapping with its approver.
- **A review UI instead of `overrides.json`.** A queue of quarantined items, side-by-side workings (e.g., the JE-003 FX recalculation), approve/reject with a reason, and four-eyes approval above materiality.
- **The full statement builder and verifier** (BS, P&L, SOCIE, indirect CF), with the invariants in the architecture doc enforced as property-based tests.
- **GL-detail ingestion**, so lineage goes below the TB row to source documents.
- **An evaluation set for the LLM parts:** labelled mapping cases from real COAs, and explanation-faithfulness checks scored over hundreds of entries, run in CI.

Where this prototype breaks at scale

- **Thousands of accounts.** The mapper scores every COA account for every unknown code. That's fine at 70 accounts, but at 5,000+ it needs an embedding index to shortlist candidates before scoring or calling an LLM.
- **Multi-entity consolidation.** There is no entity dimension, no IC matching across entities, no elimination entries, no NCI, and no translation of foreign subsidiaries to the reporting currency (CTA). JE-008 already shows the COA has no IC receivable side, so the data model needs an `entity` and `counterparty` column from day one.
- **Per-run, in-memory processing.** Large GL extracts would need streaming and a database, plus concurrency control when several preparers submit batches at once.
- **Rules are code.** A real platform needs per-client rule configuration (materiality, tolerances, FX policy) with versioning.

How I used AI tools: where they helped, where they led me wrong

I used Claude as my main implementation tool: data profiling, generating the code, tests and doc drafts, and pressure-testing accounting edge cases. I treated it like a fast junior engineer: it produced, I decided. The decisions that shaped the submission were mine:

- **Questions before building.** I sent the three clarifying questions (FX basis, the retained earnings break, JE-008 and approval authority). I rejected the first draft as too long and generic-sounding, had it cut down, and decided which lines stayed.
- **Scope under time pressure.** When I confirmed the deadline was the same day, I dropped the idea of waiting for answers and chose to proceed on conservative, documented assumptions. I picked the adjustments slice because the seeded defects (JE-002, 003, 005, 008) exercise the whole control loop.
- **Not overclaiming.** When reviewing the architecture doc against the brief, I had a "built vs designed" table added so the reviewer can see the Mapper is deterministic only and the statements are design-only.
- **LLM run.** I ran the explainer with gpt-4o-mini. All 10 explanations passed the validator, and the decisions and run_id were identical to the no-LLM run, confirming the model cannot move a decision. But the headlines were generic ("Journal Entry Accepted", no entry id or reason): the validator checks faithfulness, not usefulness. Next step: require the headline to name the entry and its main rule.

Where the AI-generated work was wrong, and how it was caught (by running the numbers and tests, not by trusting the output):

- **A silent FX fallback.** The first balance script treated the missing GBP period-end rate as 1.0 and reported +73,613. That is exactly the failure the system must prevent. The final code excludes and reports the currency instead (−338,687, GBP excluded).
- **A false-positive control.** The first sign-flip rule flagged 7310 (FX gain/loss), but it was already a credit before JE-003, and gain/loss accounts legitimately swing either way. The rule now flags only flips *caused* by the entry.
- **A misleading display.** An explanation showed the EUR rate as "1.08" (money formatting) instead of 1.082. A finance user who recomputes would lose trust; it now shows the exact rate.
- **"Obvious" mappings.** Mapping 6315→6310 looks right but makes JE-005 self-offsetting. This is why approved mappings are re-validated, not trusted.
- **The brief's framing.** The PDF calls the TB imbalance "FX rounding". It is six figures, so the system flags a basis question instead of plugging a rounding difference.

Takeaway: AI made a one-day turnaround possible, but every error above looked plausible. That is the same lesson the architecture encodes: an LLM can draft, suggest and explain, but it never owns a number or a decision.

One thing I think is being underestimated

The trial balance is already a lossy summary, and most "AI accuracy" problems here are really ownership and semantics problems. Whether the EUR rows are local or USD, whether the TB is pre- or post-closing, and why retained earnings moved 2.06M can't be inferred reliably by any model. They are facts that someone at the client owns. The product's real moat is less the model and more (a) capturing those decisions once as versioned client policy and mapping memory, and (b) ingesting GL detail so that "trace any number to source" goes beyond a TB row. Without that, audit-grade traceability stops at the TB.